# Hướng dẫn test Quarantine Management trên Swagger

Ca test này kiểm tra **đủ 13 API của Nhóm A–C** trên Docker local/dev. Luồng đi từ nạp file lỗi đến cách ly, giao người phụ trách, trả lại bản sửa còn lỗi và xác minh bản sửa đúng. Terminal dùng để chuẩn bị môi trường. Các bước gọi API thực hiện trên Swagger UI.

## 1. Chuẩn bị môi trường và tài khoản

Từ thư mục gốc repo, làm theo mục **Cài đặt và khởi chạy** trong README để cấu hình `.env`, khởi động PostgreSQL/MinIO/Redis và cấp tài khoản dịch vụ Bronze/Silver. Dùng Bash/Zsh hoặc Git Bash để chạy các lệnh bên dưới.

```
docker compose up -d postgres minio minio-init redis
docker compose ps -a

apply_sql() {
  docker compose cp "$1" postgres:/tmp/unilake-quarantine-test.sql &&
    docker compose exec -T postgres sh -c 'psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_
}
```

Kiểm tra các bảng chuẩn hóa và xuất bản Silver:

```
docker compose exec -T postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB"' <<'SQL'
SELECT to_regclass('app.canonical_identity'), to_regclass('app.identity_crosswalk'),
       to_regclass('app.crosswalk_release'), to_regclass('app.silver_run'),
       to_regclass('app.silver_snapshot'), to_regclass('app.silver_head');
SQL
```

Chỉ chạy migration S4/P5 tương ứng khi **cả ba bảng của nhóm đó đều chưa có**. Nếu chỉ thiếu một phần, đối chiếu schema trước.

```
apply_sql infra/postgres/migrations/20261015_s4_crosswalk.sql
apply_sql infra/postgres/migrations/20261015_p5_silver.sql
```

Database mới có các bảng quản lý lỗi từ DDL khởi tạo. Với volume cũ, xem hướng dẫn đồng bộ database rồi áp dụng các migration Quarantine theo thứ tự:

```
apply_sql infra/postgres/migrations/20261106_cleaning_runs.sql &&
apply_sql infra/postgres/migrations/20261106_quarantine_records.sql &&
apply_sql infra/postgres/migrations/20261106_data_quality_issues.sql &&
apply_sql infra/postgres/migrations/20261106_issue_assignees.sql
```

Khởi động backend và worker, sau đó tạo fixture:

```
docker compose up -d --build backend ingestion-worker
docker compose ps -a
curl -fsS http://localhost:8000/health

apply_sql scripts/standardization/seed_ingestion_demo.sql &&
docker compose exec -T backend python scripts/standardization/live_demo.py init-job-crosswalk &&
apply_sql scripts/standardization/seed_quarantine_test_users.sql
```

| Ghi lại | Lấy từ kết quả seed |
|---|---|
| **SOURCE_ID**, **DATASET_ID** | `data_source_id` , `dataset_id` ở dòng `academic` |
| **DEPARTMENT_ID**, **OWNER_USER_ID** | `department_id` , `app_user_id` của `qa_quarantine_owner` |
| **PEER_USER_ID**, **OUTSIDE_USER_ID** | `app_user_id` của `qa_quarantine_peer` và `qa_quarantine_outside` |

Ba tài khoản QA dùng mật khẩu của tài khoản demo `academic_staff` tại thời điểm seed. Với seed mặc định, mật khẩu demo local là `Password@123` .

# 2. Cách thao tác trên Swagger

1. Mở nhóm `auth` , chọn `POST /api/v1/auth/login` , bấm **Try it out**, nhập `{"username":"admin","password":"Password@123"}` trên môi trường dùng seed mặc định, rồi bấm **Execute**.
2. Sao chép `access_token` , bấm **Authorize** và dán token. Khi đổi vai trò, đăng nhập lại bằng đúng tài khoản, thay token trong **Authorize**. Nếu đang có token cũ, bấm **Logout** trong hộp Authorize trước khi dán token mới.
3. Với mỗi endpoint, bấm **Try it out**. Với `GET` , điền ID hoặc bộ lọc vào đúng ô. Với `POST` hoặc `PATCH` , điền ID vào ô đường dẫn nếu có và dán JSON vào **Request body**. Bấm **Execute**, kiểm tra mã HTTP và JSON trong **Server response**.
4. Các tên viết hoa như **RUN_GỐC** là ký hiệu để ghi chú. Dùng ID thực tế trong response, không dán nguyên các tên này vào Swagger.

# 3. Nạp Excel lỗi và kiểm tra đủ 7 API Nhóm A–B

Dùng **admin**, mở nhóm `ingestion` và chọn `POST /api/v1/ingestion/jobs` . Thay hai ID minh họa bằng **SOURCE_ID** và **DATASET_ID**.

```
{
  "data_source_id": 6,
  "dataset_id": 5,
  "source_table": "04-bad-date.xlsx"
}
```

- Kỳ vọng **202**. Lưu `ingestion_job_id` làm **JOB_GỐC**.
- Gọi `GET /api/v1/ingestion/jobs/{job_id}` với **JOB_GỐC** đến khi `SUCCESS`. File có thể nạp Bronze thành công vì ngày sai được phát hiện ở Cleaning.
- Mở nhóm `pipeline`, gọi `POST /api/v1/pipeline/cleaning/ingestion-jobs/{ingestion_job_id}` với **JOB_GỐC** và body `{"session":"qa-academic-01","base_snapshot":"active"}`. Nếu test lại, chọn session mới và dùng cùng session trong cả ca. Kỳ vọng **200**, `QUARANTINED`, 1 dòng cách ly. Lưu `run_id` làm **RUN_GỐC**.

Mở nhóm `cleaning-runs` và `quarantine-records`. Tất cả các ca dưới đây dùng admin và kỳ vọng **200**.

| Ca | Endpoint trên Swagger | Ô cần điền | Kết quả cần kiểm tra |
|---|---|---|---|
| A1 | `GET /api/v1/cleaning-runs` | `ingestion_job_id` = **JOB_GỐC** | Có lượt vừa tạo |
| A2 | `GET /api/v1/cleaning-runs/{run_id}` | `run_id` = **RUN_GỐC** | Cleaning `COMPLETED`, publication `BLOCKED`, 0 dòng xuất bản, có báo cáo |
| A3 | `GET /api/v1/cleaning-runs/{run_id}/report` | `run_id` = **RUN_GỐC** | Báo cáo có lỗi `DATE_FORMAT_INVALID` |
| B1 | `GET /api/v1/quarantine-records` | `cleaning_run_id` = **RUN_GỐC** | Đúng 1 hồ sơ. Lưu `record_id` làm **RECORD_ID** |
| B2 | `GET /api/v1/quarantine-records/{record_id}` | `record_id` = **RECORD_ID** | Có mã lỗi, vị trí và phiên bản quy tắc |
| B3 | `GET /api/v1/quarantine-records/{record_id}/source` | `record_id` = **RECORD_ID** | Sheet `Data`, dòng Excel **2**, trường lỗi `tu`, đúng phiên bản Bronze |
| B4 | `GET /api/v1/quarantine-records/summary` | `cleaning_run_id` = **RUN_GỐC** | Tổng 1 hồ sơ và 1 hồ sơ có mã `DATE_FORMAT_INVALID` |

# 4. Kiểm tra 6 API Nhóm C

Mở nhóm `data-quality-issues`. Với các URL có `{issue_id}`, điền **ISSUE_ID** vào ô `issue_id`.

## C1. Tạo yêu cầu

Dùng **admin**, gọi `POST /api/v1/data-quality-issues`. Thay số phòng và UUID bằng **DEPARTMENT_ID** và **RECORD_ID**.

```
{
  "title": "QA: ngày bắt đầu học kỳ sai định dạng",
  "description": "Đối chiếu ngày trong file đào tạo mẫu",
  "department_id": 3,
  "quarantine_record_ids": ["3cba220a-098f-4b93-9e24-eb95ade1879f"]
}
```

Kỳ vọng **201**, trạng thái `ASSIGNED`, `assignee_user_id=null`. Lưu `issue_id` làm **ISSUE_ID**.

## C2. Giao người phụ trách

Dùng **admin**, gọi `PATCH /api/v1/data-quality-issues/{issue_id}/assignment` với `{"assignee_user_id":42}`. Thay 42 bằng **OWNER_USER_ID**. Kỳ vọng **200**, đúng ID người phụ trách, lịch sử có `ASSIGNEE_CHANGED`. Thử ID **OUTSIDE_USER_ID**, kỳ vọng **422**.

## C3. Danh sách yêu cầu

Dùng **admin**, gọi `GET /api/v1/data-quality-issues` với `assignee_user_id` = **OWNER_USER_ID** và `status` = `ASSIGNED`. Kỳ vọng **200** và thấy **ISSUE_ID** trong danh sách.

## C4. Chi tiết và quyền xem

Gọi `GET /api/v1/data-quality-issues/{issue_id}` với token của từng tài khoản:

- `qa_quarantine_owner` và `qa_quarantine_peer`: **200**, thấy hồ sơ lỗi và lịch sử.
- `qa_quarantine_outside`: **404** vì ngoài phạm vi phòng ban.

## C5. Báo đã sửa với bản còn lỗi

1. Dùng **admin**, lặp lại bước nạp và Cleaning với `04-bad-date.xlsx` để tạo **JOB_LỖI_MỚI** và **RUN_LỖI_MỚI**. Job phải `SUCCESS` ở Bronze và Cleaning phải `QUARANTINED`.
2. Gọi `POST /api/v1/data-quality-issues/{issue_id}/resolution` với `{"ingestion_job_id":123,"note":"Nộp lại bản sửa để kiểm tra"}`, thay 123 bằng **JOB_LỖI_MỚI**.
3. Thử bằng **peer** và **admin** trước. Cả hai phải nhận **403**. Đổi sang **owner**, gửi cùng payload, kỳ vọng **200** và `PENDING_VERIFICATION`.

## C6. Xác minh nhánh từ chối rồi nhánh đạt

Dùng **admin**, gọi `POST /api/v1/data-quality-issues/{issue_id}/verification` với body dưới đây. Thay UUID bằng **RUN_LỖI_MỚI**.

```
{
  "cleaning_run_id": "32840586-3d8e-4aeb-9b7e-af4388f4347d",
  "verified": false,
  "note": "Ngày ở bản nộp vẫn sai"
}
```

Kỳ vọng **200**, trở về `ASSIGNED` và có sự kiện `VERIFICATION_REJECTED`. Nếu owner gửi lại chính **JOB_LỖI_MỚI**, API resolution trả **422** vì lần nộp tiếp theo cần job mới hơn.

Tiếp theo:

1. **Admin** nạp file `02-academic.xlsx` với cùng source/dataset, đợi `SUCCESS` rồi chạy Cleaning cùng session. Lưu ID làm **JOB_ĐÃ_SỬA** và **RUN_ĐÃ_SỬA**. Kỳ vọng `PUBLISHED`, 1 dòng lên Silver, 0 dòng cách ly và có `silver_snapshot_id`. Gọi lại A3 với **RUN_ĐÃ_SỬA** phải nhận **404** và mã `CLEANING_REPORT_NOT_FOUND`.
2. **Owner** gọi lại C5 với **JOB_ĐÃ_SỬA**, kỳ vọng `PENDING_VERIFICATION`.
3. **Admin** đối chiếu học kỳ `2026-1` và ngày bắt đầu `2026-09-01` trong Silver. Sau đó gọi lại C6 với **RUN_ĐÃ_SỬA** và **RECORD_ID gốc**:

```
{
  "cleaning_run_id": "32840586-3d8e-4aeb-9b7e-af4388f4347d",
  "verified": true,
  "note": "Đã đối chiếu mã học kỳ và ngày bắt đầu trong Silver",
  "confirmed_record_ids": ["3cba220a-098f-4b93-9e24-eb95ade1879f"]
}
```

Kỳ vọng **200**, `VERIFIED`. Gọi lại C4 để kiểm tra `resolution_job_id`, `verification_run_id` và lịch sử có cả lần từ chối lẫn lần xác minh.

Nếu cần xem file Silver, lấy key manifest bằng truy vấn sau trong psql hoặc DBeaver, thay ID bằng `silver_snapshot_id` của lượt đã sửa:

```
SELECT manifest_ref::json ->> 'key' AS manifest_key
FROM app.silver_snapshot
WHERE snapshot_id = '<SNAPSHOT_ID>';
```

Mở manifest trong bucket Silver ở MinIO Console, tải file theo `entities.semester.object.key` rồi đọc hai cột cần đối chiếu:

```
python -c 'import pyarrow.parquet as pq; import sys; print(pq.read_table(sys.argv[1], columns=["semes
```

# 5. Ghi kết quả và xử lý lỗi môi trường

Đánh dấu **A1–A3, B1–B4 và C1–C6**. Ghi lại các ID job, lượt, hồ sơ và yêu cầu, cùng mã HTTP của ca quyền và ca từ chối. Ca test đạt khi người không được giao bị chặn, lịch sử giữ lần sửa chưa đạt và bản sửa hợp lệ được xác minh.

| Hiện tượng | Kiểm tra |
|---|---|
| Job kẹt `QUEUED` | `docker compose logs --tail 50 ingestion-worker` |
| Không đọc được file | Tên XLSX, ID nguồn và thư mục mount `/data/standardization/live-demo` |
| Lỗi đọc báo cáo hoặc ghi Silver | MinIO và tài khoản dịch vụ Bronze/Silver |
| File đúng vẫn không lên Silver | Migration S4/P5, crosswalk demo và session dùng khi chạy Cleaning |
| Người được giao nhận 403 | Token đang dùng, `assignee_user_id`, vai trò và phân công còn hiệu lực |
| Cần chạy lại toàn bộ | Tạo job mới và chọn session mới để đối chiếu riêng |

Báo cáo và sơ đồ cho team · Script tài khoản QA